

Stack in Python

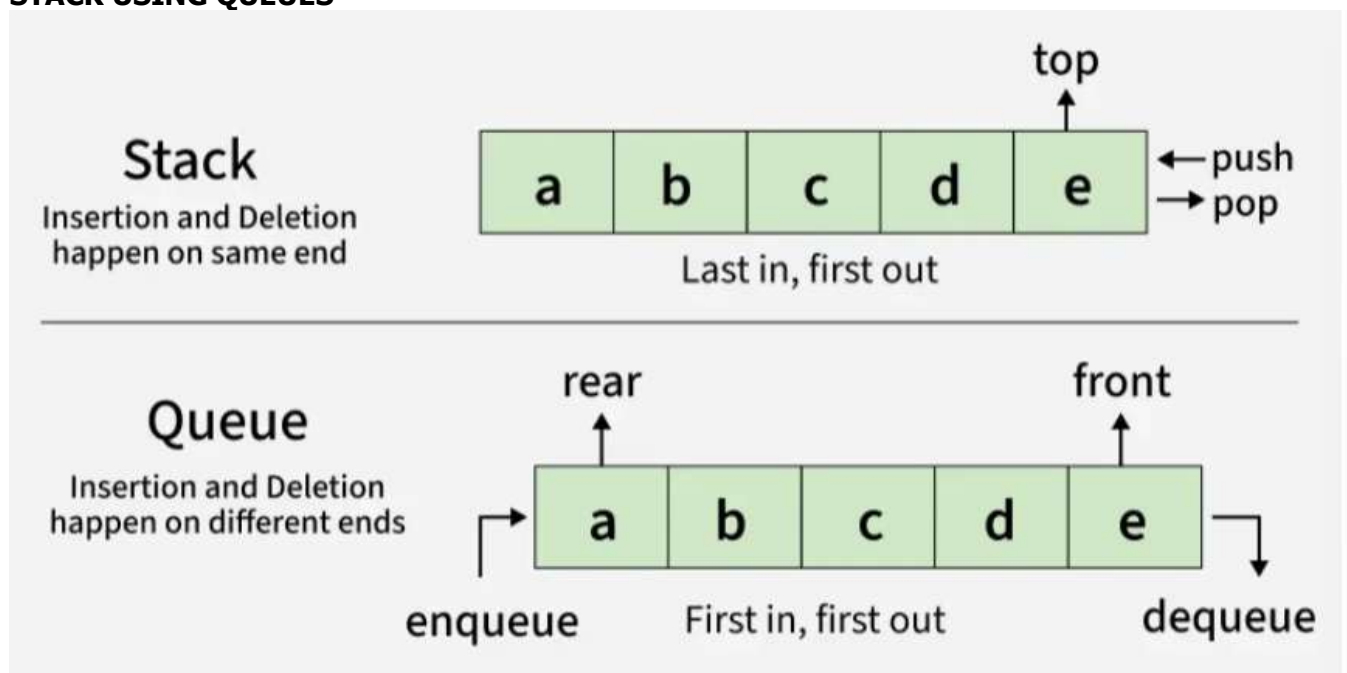
Tuesday, May 5, 2026 9:49 AM

A stack is a linear data structure that follows LIFO. This means that the last element added is the first one to be removed. In a stack, both insertion and deletion happen at the same end, which is called the TOS.

The stack can be implemented in multiple ways:

- 1) Using a list: the append method adds an element to the end of the list, the pop method removes the last element from the list.
- 2) Using collections.deque: the append method adds an element to the right end of the deque. The pop method removes the element from the right end. It is faster than lists.
- 3) Using queue.LifoQueue: it is thread-safe, making it suitable for multi-thread applications. It is generally slower than list and deque.

STACK USING QUEUES



The stack should support the following operations:

- Push: insert an element on TOS
 - Pop: remove an element from the TOS, if the stack is empty, do nothing.
 - Top: return the element at the TOS.
 - Size: return the number of elements in the stack.
- 1) Using two queues: push in $O(n)$ and pop in $O(1)$. Always keep the newly inserted element at the front of $q1$, so that both pop and top can access it.
 - Push: enqueue the new element into $q2$, move all elements from $q1$ to $q2$, swap the names.
 - Pop: dequeue from the front of $q1$, which is the TOS.
 - Top: return the front element.
 - Size: number of elements in $q1$.

```
class myStack:
    def __init__(self):
        self.q1 = deque()
        self.q2 = deque()

    def push(self, x):
        # Push x first in empty q2
```

```

        self.q2.append(x)

        # Push all the remaining
        # elements in q1 to q2.
        while len(self.q1) != 0:
            self.q2.append(self.q1[0])
            self.q1.popleft()

        # swap the names of two queues
        self.q1, self.q2 = self.q2, self.q1

    def pop(self):
        # if no elements are there in q1
        if len(self.q1) == 0:
            return
        self.q1.popleft()

    def top(self):
        if len(self.q1) == 0:
            return -1
        return self.q1[0]

    def size(self):
        return len(self.q1)

if __name__ == '__main__':
    st = myStack()
    st.push(1)
    st.push(2)
    st.push(3)

    print(st.top())
    st.pop()
    print(st.top())
    st.pop()
    print(st.top())

    print(st.size())

```

2) Using two queue: push in $O(1)$ and pop in $O(n)$: the idea is to maintain the stack LIFO behavior.

- Push: the element is enqueued into q1.
- Pop: move elements from q1 to q2 until one element is left in q1, remove this last element from q1, swap the names.
- Top: move elements from q1 to q2 until one element is left, store this last element, then move it to q2 instead of discarding it, swap the queues.

```
from collections import deque
```

```

class myStack:
    def __init__(self):
        self.q1 = deque()
        self.q2 = deque()

```

```

    # insert element
    def push(self, x):
        self.q1.append(x)

```

```

    # remove top element

```

```
def pop(self):
    if not self.q1:
        return
```

```
    while len(self.q1) != 1:
        self.q2.append(self.q1.popleft())
    self.q1.popleft()
    self.q1, self.q2 = self.q2, self.q1
```

```
# return top element
def top(self):
    if not self.q1:
        return -1
```

```
    while len(self.q1) != 1:
        self.q2.append(self.q1.popleft())
    temp = self.q1.popleft()
    self.q2.append(temp)
    self.q1, self.q2 = self.q2, self.q1
    return temp
```

```
# return current size
def size(self):
    return len(self.q1)
```

```
if __name__ == '__main__':
    st = myStack()
    st.push(1)
    st.push(2)
    st.push(3)
```

```
    print(st.top())
    st.pop()
    print(st.top())
    st.pop()
    print(st.top())
    print(st.size())
```

3) Using single queue: push in $O(n)$ and pop in $O(1)$. The main idea is to always keep the most recently pushed element at the front of the queue, so that top and pop work in $O(1)$ time.

- Push: insert the new element at the back of the queue, rotate the queue by repeatedly removing the front element and pushing it back until the new element comes to the front. After rotation, the newest element will be at the front acting as TOS.
- Top: return the element at the front
- Pop: dequeue the front element of the queue.

```
from collections import deque
```

```
class myStack:
    def __init__(self):
        self.q = deque()
```

```
# push element on top
def push(self, x):
    self.q.append(x)
```

```
    sz = len(self.q)
    for i in range(sz - 1):
```

```
self.q.append(self.q[0])
self.q.popleft()
```

```
# remove top element
def pop(self):
    if self.q:
        self.q.popleft()
```

```
# return top element
def top(self):
    if not self.q:
        return -1
    return self.q[0]
```

```
# return current size
def size(self):
    return len(self.q)
```

```
if __name__ == '__main__':
    st = myStack()
    st.push(1)
    st.push(2)
    st.push(3)
```

```
print(st.top())
st.pop()
print(st.top())
st.pop()
print(st.top())
print(st.size())
```

STACK USING LINKED LISTS

A stack can be implemented using linked lists where we maintain:

- A node class that contains data, to store the element, and next, a pointer to the next node of the stack.
- A pointer top that always points to the current TOS.

Node structure

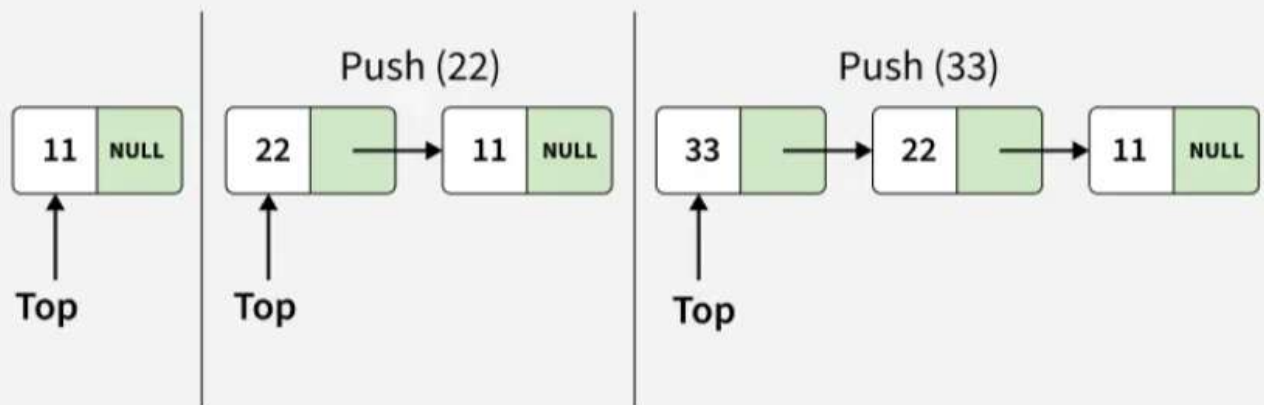
```
class Node:
    def __init__(self, x):
        self.data = x
        self.next = None
```

Stack class

```
class myStack:
    def __init__(self):
        # initially stack is empty
        self.top = None
```

- Push: a new node is created with the given value, the new node's next pointer is set to the current top, the top pointer is updated to point to this new node.

Push elements in the stack



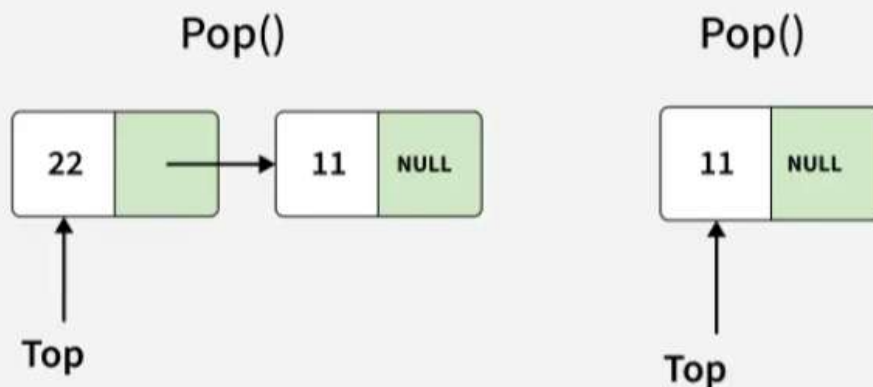
```
def push(self, x):
    temp = Node(x)
    temp.next = self.top
    self.top = temp
```

Time complexity: $O(1)$

Space complexity: $O(1)$

- Pop: we store the current top node in a temporary pointer, move the top pointer to the next node and delete the temporary node.

Pop elements from the stack



```
def pop(self):
    if self.top is None:
        print("Stack Underflow")
        return -1
```

```
    temp = self.top
    self.top = self.top.next
    val = temp.data
```

```
del temp
return val
```

Time complexity: $O(1)$

Space complexity: $O(1)$

- Peer or Top:

```
def peek(self):
    if self.top is None:
        print("Stack is Empty")
        return -1
    return self.top.data
```

Node structure

```
class Node:
    def __init__(self, x):
        self.data = x
        self.next = None
```

Stack implementation using linked list

```
class myStack:
    def __init__(self):
        # initially stack is empty
        self.top = None
        self.count = 0
```

push operation

```
def push(self, x):
    temp = Node(x)
    temp.next = self.top
    self.top = temp
```

```
self.count += 1
```

pop operation

```
def pop(self):
    if self.top is None:
        print("Stack Underflow")
        return -1
    temp = self.top
    self.top = self.top.next
    val = temp.data
```

```
self.count -= 1
return val
```

peek operation

```
def peek(self):
    if self.top is None:
        print("Stack is Empty")
        return -1
    return self.top.data
```

check if stack is empty

```
def isEmpty(self):
    return self.top is None
```

size of stack

```
def size(self):
```

```
return self.count
```

```
if __name__ == "__main__":  
    st = myStack()
```

```
# pushing elements  
st.push(1)  
st.push(2)  
st.push(3)  
st.push(4)
```

```
# popping one element  
print("Popped:", st.pop())
```

```
# checking top element  
print("Top element:", st.peek())
```

```
# checking if stack is empty  
print("Is stack empty:", "Yes" if st.isEmpty() else "No")
```

```
# checking current size  
print("Current size:", st.size())
```

Advantages:

Advantages of Stack using Linked List



Dynamic Size

Can grow or shrink as needed without a fixed capacity.



Efficient Memory

Allocates memory only when elements are added.



No Overflow

Overflow occurs only if system memory is exhausted.



Easy Operations

Push and pop are simple pointer updates.



Flexible

No need for resizing like in arrays.